



Enhancing the Kubernetes Platform with a Load-Aware Orchestration Strategy

Angelo Marchese¹ · Orazio Tomarchio¹

Received: 5 October 2024 / Accepted: 16 January 2025
© The Author(s) 2025

Abstract

Microservice architecture is increasingly favored for building large-scale applications designed for deployment in distributed and resource-constrained cloud-to-edge computing environments. As a cloud-native approach, microservices are particularly powerful due to their loosely coupled, independently deployable, and scalable characteristics. These features enable distributed deployment and flexible integration across robust cloud data centers and heterogeneous, often resource-limited, edge nodes. To fully leverage these advantages, there is a need for advanced placement algorithms that can optimize application performance by utilizing the strengths of microservices. In response to these challenges, we propose to extend Kubernetes with a load-aware orchestration strategy, which enhances its capability to deploy microservice applications within shared clusters characterized by dynamic resource usage patterns. Our approach orchestrates applications dynamically, based on real-time resource usage, allowing for continuous adjustment of their placement to match evolving conditions. Evaluation of a prototype system in a testbed environment demonstrates that this approach offers substantial benefits compared to the standard Kubernetes scheduler.

Keywords Microservice architecture · Container technology · Kubernetes scheduler · Kubernetes descheduler · Cloud continuum

Introduction

Orchestrating modern applications, such as those in the Internet of Things (IoT), data analytics, video streaming, process control, and augmented reality services, presents a significant challenge [1–3]. These applications demand stringent quality of service (QoS) requirements, particularly in scalability, fault tolerance, availability, response time, and throughput. To meet these demands, the microservices architecture has become the leading approach for designing and implementing such applications. This architecture involves breaking down applications into multiple microservices that work together to fulfill user requests.

Cloud Computing provides a reliable and scalable environment for executing these applications, while the recent adoption of Edge Computing enables workloads to be processed closer to the end user [4, 5]. The combination of Cloud and Edge infrastructures creates a Cloud-to-Edge continuum, a unified environment for running distributed, microservices-based applications. However, orchestrating these applications across this continuum is complex due to the heterogeneity of computational resources between Cloud and Edge nodes, as well as the competition among microservices for these resources [6–9].

Kubernetes¹ is a widely adopted orchestration platform that supports the deployment, scheduling, and management

✉ Angelo Marchese
angelo.marchese@phd.unict.it

Orazio Tomarchio
orazio.tomarchio@unict.it

¹ Dept. of Electrical Electronic and Computer Engineering,
University of Catania, Catania, Italy

¹ <https://kubernetes.io>.

of containerized applications [10]. Various Kubernetes distributions are maintained by major Cloud providers, and more recently, Edge-oriented distributions like K3s,² KubeEdge³ and MicroK8s⁴ have been introduced. However, the default static orchestration policy of Kubernetes has limitations when handling complex microservices-based applications that share the same node cluster. Specifically, Kubernetes does not account for the runtime resource usage of the cluster infrastructure and the microservices running on it during scheduling. This can lead to increased resource contention and degraded application performance, particularly in terms of response time. At the same time, reducing resource contention among microservices would require a higher number of nodes, resulting in lower resource utilization and increased infrastructure costs.

To overcome these limitations, by extending our previous work presented in [11], we propose to enhance the Kubernetes platform with a load-aware orchestration strategy better suited for deploying microservices applications with dynamic resource usage patterns on shared node clusters. Our approach enhances Kubernetes by implementing a dynamic orchestration and scheduling strategy that: 1) takes into account the runtime resource usage patterns

of applications when determining the placement of each microservice, and 2) continuously adjusts application placement based on the changing states of the infrastructure and applications. While various factors such as application dependability and availability can be optimized, the primary goal of our approach in this work is to improve application performance and increase cluster resource utilization by allowing to colocate more applications on the same cluster while at the same time maintaining acceptable response times.

The rest of the paper is organized as follows. Section “[Background](#)” provides some background information about the Kubernetes platform while section “[Motivation](#)” discusses in more detail some of its limitations that motivate our work. In section “[Proposed framework](#)” the proposed framework is presented, providing some implementation details of its components, while section “[Performance evaluation](#)” provides results of our prototype evaluation in a test-bed environment. Section “[Related work](#)” examines some related works and, finally, section “[Conclusions and future work](#)” concludes the work.

Background

Kubernetes Scheduler

Listing 1 Example of Kubernetes Deployment with resource requests [11]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:latest
        resources:
          requests:
            cpu: 1
            memory: 64Mi
```

² <https://k3s.io/>.

³ <https://kubededge.io>.

⁴ <https://microk8s.io>.

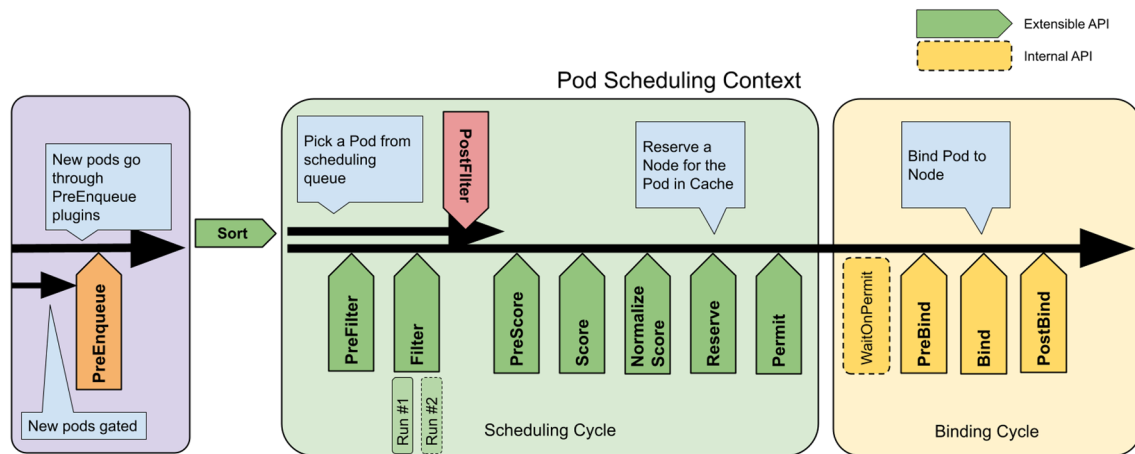


Fig. 1 Kubernetes scheduling framework [11]

Kubernetes is a container orchestration platform that automates the lifecycle management of distributed applications across large-scale node clusters [10, 12]. A typical Kubernetes cluster consists of a control plane and a set of worker nodes. The control plane manages the cluster's operations through various services running on one or more master nodes, while the worker nodes provide the execution environment for containerized workloads. In Kubernetes, the core deployment units are known as *Pods*, which contain one or more containers and are managed by *Deployments*. In a microservices-based architecture, each Deployment corresponds to a specific microservice, with the Pods managed by that Deployment representing individual instances of the microservice. Application developers can configure several aspects of a Deployment, including the resource requests⁵ for Pods, which specify the computational resources to be reserved for them when they are scheduled on worker nodes. Earlier versions of Kubernetes allowed users to specify only CPU cores and memory as resource requirements for Pods. However, with the introduction of the extended resources mechanism⁶ in the latest versions, it is now possible to specify resource requirements for additional resources such as network and disk bandwidth. For example, Listing 1 illustrates a Deployment with specified CPU and memory Pod resource requests.

The *kube-scheduler*⁷ is a crucial component of the Kubernetes control plane, responsible for selecting a cluster node for each Pod based on the Pod's requirements and the available resources on the cluster nodes. Each scheduling attempt for a Pod follows a multi-phase process, as depicted

in Fig. 1. The main phases—sorting, filtering, and scoring—comprise the core execution logic of the scheduler. These phases are implemented through various plugins, which may also handle additional phases beyond the core ones. During the sorting phase, the scheduler establishes an order for the Pod scheduling queue. In the filtering phase, each plugin evaluates cluster nodes to determine whether they meet specific constraints required by the Pod. The result of this phase is a list of candidate nodes deemed suitable for hosting the Pod. In the scoring phase, each plugin assigns a score to each candidate node based on specific criteria, with the final score determined by the weighted sum of the scores from all scoring plugins. The Pod is then assigned to the node with the highest final score. If multiple nodes share the highest score, one is selected at random. The Kubernetes scheduler is designed for extensibility, allowing custom plugins to be registered at each scheduling phase, providing flexibility to accommodate various scheduling needs and strategies.

Among the default scheduler plugins in Kubernetes, the *NodeResourcesFit* plugin handles both the filtering and scoring phases. It ensures that nodes without sufficient computational resources to meet a Pod's resource requests are filtered out during the filtering phase. Additionally, the *NodeResourcesBalancedAllocation* plugin is responsible for the scoring phase. The *NodeResourcesFit* plugin assigns scores to nodes based on mutually exclusive strategies:

- *LeastAllocated*: this strategy prioritizes nodes with the lowest ratio between the weighted sum of requested resources from all Pods running on the node (including the resource requests of the Pod to be scheduled) and the total allocatable resources available on the node. The goal is to achieve balanced resource utilization across the cluster nodes.

⁵ <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers>.

⁶ <https://kubernetes.io/docs/tasks/administer-cluster/extended-resource-node/>.

⁷ <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler>.

- *MostAllocated*: this strategy favors nodes with the highest ratio between the weighted sum of requested resources from all Pods running on the node (including the resource requests of the Pod to be scheduled) and the total allocatable resources on the node. The aim here is to maximize resource utilization on individual nodes, which can help reduce the overall number of nodes needed to run the workload efficiently.
- *RequestedToCapacityRatio*: this strategy allocates Pods in a way that maintains a specified ratio between the total requested resources of all Pods running on a node and the node's total allocatable resources. This helps in achieving a desired balance of resource consumption relative to the node's capacity.

The NodeResourcesBalancedAllocation plugin prioritizes nodes that would achieve more balanced resource usage if a Pod were scheduled there. It assigns higher scores to nodes where the ratios of requested resources to allocatable resources are closer across different resource types. The goal of this plugin is to encourage an even distribution of resource allocation, preventing scenarios where one resource is fully allocated while another remains underutilized. By promoting balanced resource usage, this plugin helps optimize the overall efficiency and performance of the cluster.

Kubernetes Descheduler

The Kubernetes scheduler's placement decisions are influenced by the cluster's state at the time a new Pod is scheduled. Given the dynamic and constantly evolving nature of Kubernetes clusters, optimizing these placement decisions can be challenging. However, the initial scheduling of Pods can be improved by taking into account these dynamic factors.

Several factors may necessitate the migration of a Pod from one node to another, such as:

- Node under-utilization or over-utilization: adjusting the distribution of Pods to optimize resource usage across the cluster.
- Changes in Pod or node affinity requirements: ensuring Pods are located on nodes that meet updated affinity or anti-affinity rules.
- Node events: responding to events like node failures or the addition of new nodes, which may require redistributing Pods to maintain stability and performance.

By considering these factors, Kubernetes can make more informed and adaptive scheduling decisions, leading to better overall cluster performance and resource utilization.

To address the dynamic nature of Kubernetes clusters and enhance the placement of Pods, a descheduler component has been proposed as a Kubernetes sub-project.⁸ This component is designed to evict running Pods, enabling them to be rescheduled onto more suitable nodes. It's important to note that the descheduler itself does not handle the rescheduling of evicted Pods; instead, it relies on the default Kubernetes scheduler to perform this task. The descheduler's behavior is configurable, with a set of default eviction plugins that can be enabled or disabled based on the desired policy. It includes a common eviction configuration at the top level, as well as specific configuration options for the Default Evictor plugin. Figure 2 illustrates the various default eviction plugins available in the Kubernetes descheduler at the time of writing. Like the Kubernetes scheduler, the descheduler is designed to be extensible, allowing for the integration of custom eviction plugins to meet specific needs.

Among the default eviction plugins, the *LowNodeUtilization* and *HighNodeUtilization* plugins manage Pod eviction based on the resource utilization levels of cluster nodes. The *LowNodeUtilization* plugin identifies underutilized nodes and evicts Pods from other nodes, with the expectation that the evicted Pods will be rescheduled onto these underutilized nodes to achieve a more balanced resource distribution. On the other hand, the *HighNodeUtilization* plugin focuses on nodes with high resource utilization. It evicts Pods from these nodes with the intention of consolidating them onto fewer nodes, thereby freeing up resources and potentially reducing the overall node count needed to run the workloads efficiently.

Motivation

The Kubernetes platform is widely adopted for orchestrating a diverse range of applications, from traditional web services to modern applications such as the Internet of Things (IoT), data analytics, AI training, augmented reality, and video streaming services. These applications typically consist of complex microservices architectures and have stringent requirements for response latency. Additionally, the infrastructure hosting these applications has evolved from traditional Cloud-only environments to Cloud-to-Edge continuum environments, enabling workloads to be executed closer to end users. Edge environments consist of geographically distributed nodes with significant heterogeneity in computational and network resources. Unlike the more stable Cloud data centers, Edge infrastructure is dynamic and prone to frequent node failures and network partitions

⁸ <https://github.com/kubernetes-sigs/descheduler>.

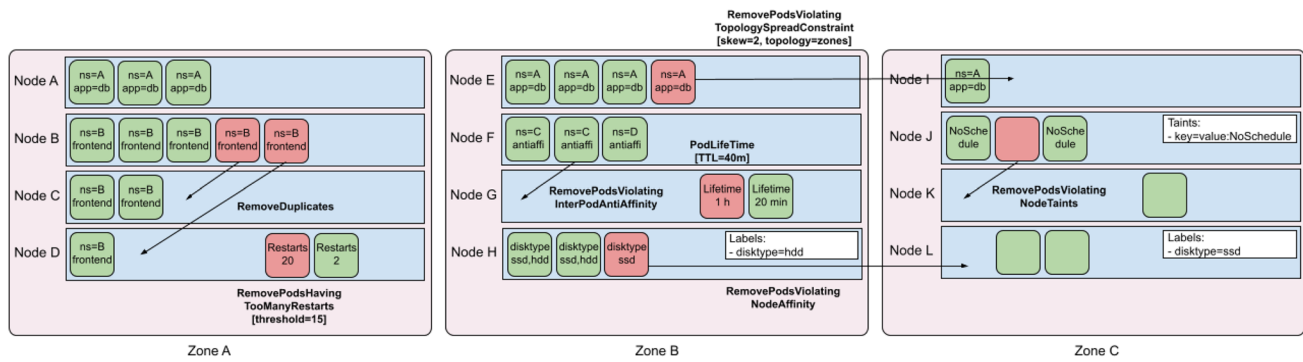


Fig. 2 Kubernetes descheduling strategies [11]

[6]. The primary challenge in orchestrating applications on such infrastructure is to ensure their expected performance while minimizing infrastructure costs. In this context, the default Kubernetes orchestration strategy is not well-suited. Specifically, we identify two major limitations in the default Kubernetes scheduling and descheduling strategies that can impact both performance and cost optimization goals.

The first limitation derives from Kubernetes not considering the current state of the infrastructure when making Pod scheduling and descheduling decisions. Specifically, the scheduler and descheduler components do not monitor real-time CPU, memory, network, and disk bandwidth utilization on each node. Instead, they evaluate a node's resource utilization by summing the resource requests of all Pods running on that node. This approach can lead to differences between estimated and actual runtime resource usage. If resource usage is underestimated, more Pods may be scheduled on the same node, leading to increased resource contention and reduced overall application performance. Conversely, if resource usage is overestimated, fewer Pods are scheduled on the node, resulting in lower node utilization and increased infrastructure costs caused by the higher number of nodes required to run the application.

The second limitation is that Kubernetes does not consider the real-time state of the application when making Pod scheduling and descheduling decisions. Instead of accounting for the dynamic CPU, memory, network, and disk bandwidth utilization of each microservice, the scheduler and descheduler rely on statically defined Pod resource requirements. This forces application developers to predict resource requirements for each microservice in advance, a challenging task given that these requirements are dynamic and heavily influenced by runtime load and user request distribution. Defining Pod resource requirements before runtime can lead to inefficient scheduling decisions, ultimately reducing application performance. Overestimating resource requirements decreases the likelihood of scheduling Pods on resource-constrained Edge nodes closer to end users and

would reduce cluster resources utilization by requiring a higher number of nodes to run the application. Conversely, underestimating resource requirements increases Pod density on cluster nodes, leading to greater interference. Therefore, considering real-time resource usage of each microservice instead of statically defined Pod resource requirements, is crucial for developing an effective scheduling strategy that minimizes QoS violations in complex microservices applications while also reducing infrastructure costs.

Proposed Framework

Overall Design

Considering the limitations outlined in section “[Motivation](#)”, this work proposes an extension to the Kubernetes platform aimed at incorporating a load-aware scheduling and descheduling strategy. This enhancement makes the platform more suitable for orchestrating microservices-based applications with dynamic resource usage patterns. The core concept of this approach is that the scheduling and descheduling processes for complex microservices applications should consider both the dynamic state of the infrastructure and the real-time resource requirements of the microservices.

While various scheduling and descheduling strategies can be developed based on specific optimization goals, our focus in this work is on minimizing runtime shared resource interference among microservices Pods. By doing so, we aim to enhance application response times and to improve node resource utilization by increasing the number of applications that can be colocated on the same cluster while still maintaining acceptable response times. A key distinction between our proposed approach and the default Kubernetes scheduler and descheduler components is the use of real-time telemetry data for microservices resource usage profiling. Unlike statically defined Pod resource requests, this dynamic profiling reduces the burden on application developers to predict

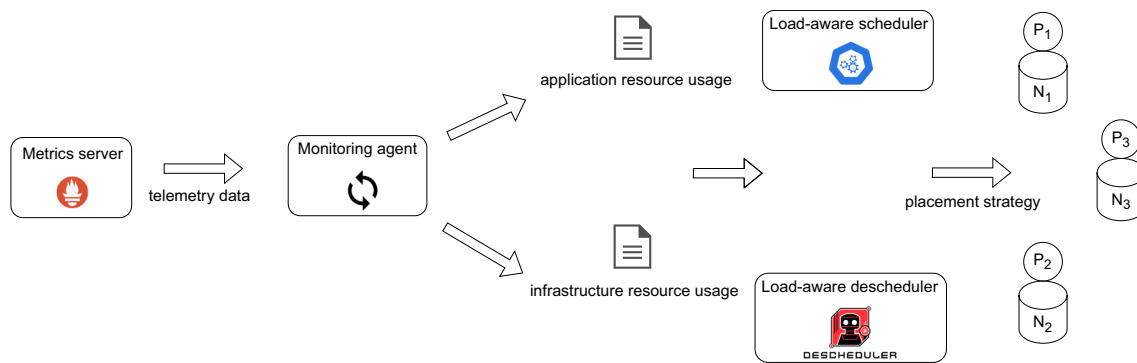


Fig. 3 General architecture of the proposed framework

microservices resource usage in advance when defining Pod resource requirements. This approach not only streamlines the scheduling process but also ensures that resource allocation is more responsive to the actual requirements of the applications, leading to better performance and resource utilization.

The proposed approach is illustrated in Fig. 3, which presents a general architecture overview. Continuous monitoring of runtime infrastructure and application microservices resource usage is facilitated by a *metrics server*. This server collects telemetry data on CPU, memory, network, and disk bandwidth resources for both cluster nodes and Pods within the cluster.

A *monitoring agent* uses this infrastructure telemetry data to annotate each cluster node with resource usage information, providing a real-time view of the cluster state. Similarly, the agent utilizes application telemetry data to annotate each application microservice Deployment with resource usage information, creating a real-time view of the application state.

The *load-aware scheduler* leverages these runtime cluster and application states to determine the optimal placement for each application Pod. Meanwhile, the *load-aware descheduler* is responsible for initiating Pod rescheduling actions if more efficient placement decisions can be made. Further details on the components of the proposed approach are provided in the following subsections.

Monitoring Framework

To support the telemetry-driven approach behind the proposed scheduling and descheduling strategies, a robust monitoring framework is required. This framework is designed to collect all essential parameters necessary for maintaining a real-time view of both the infrastructure and the hosted applications. Figure 4 illustrates the key components of this framework, including elements that operate within the

Kubernetes control plane and those deployed on the worker nodes.

```
apiVersion: unict.it/v1alpha1
kind: Cluster
metadata:
  name: cluster
spec:
  runPeriod: 15
  nodeSelector: {}
  rangeWidth: 5m

apiVersion: unict.it/v1alpha1
kind: AppGroup
metadata:
  name: appgroup
spec:
  runPeriod: 15
  podSelector: {}
  rangeWidth: 5m
```

Listing 2: Cluster and AppGroup custom resources

The monitoring agent is a Kubernetes operator written in the Java language using the Quarkus Operator SDK⁹ and runs as a Deployment in the Kubernetes control plane. Being a Kubernetes operator, it is triggered by Kubernetes custom resources, specifically the *Cluster* and *AppGroup* custom resources whose schemas are shown in Listing 2.

A *Cluster* resource includes a *spec* property with three sub-properties: *runPeriod*, *nodeSelector* and *rangeWidth*. The *runPeriod* property specifies the interval, in seconds, between consecutive executions of the custom resource processing logic. The *nodeSelector* property serves as a filter, selecting the specific nodes in the cluster that the agent should monitor. During each iteration, the agent retrieves the list of nodes that meet the conditions defined by the *nodeSelector* property from the Kubernetes API server. For each node n , the agent calculates the runtime total usage of CPU, memory, network, and disk bandwidth resources, denoted as cpu_n , mem_n , net_n , and $disk_n$, respectively.

These values are derived from metrics retrieved by the agent from a Prometheus¹⁰ metrics server. The Prometheus server gathers these metrics from node exporters,¹¹ which are deployed as *DaemonSets* on each cluster node. A *DaemonSet* is a Kubernetes resource that ensures identical Pods are running on each cluster node. The metrics, stored on

⁹ <https://github.com/quarkiverse/quarkus-operator-sdk>.

¹⁰ <https://prometheus.io/>.

¹¹ https://github.com/prometheus/node_exporter.

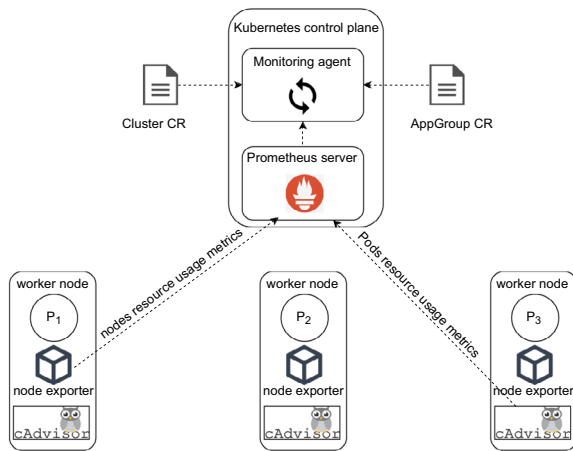


Fig. 4 Monitoring framework

the Prometheus server, are time series collections of measurements that the Prometheus server queries from the node exporters at a configurable interval of qt seconds.

Table 1 shows the *PromQL* queries executed by the agent on the Prometheus server to retrieve the runtime usage of each node resource type. Each value is calculated as the average of the corresponding metric over a configurable time duration, which is determined by the *rangeWidth* property of the Cluster custom resource.

The parameters cpu_n , mem_n , net_n , and $disk_n$ are then assigned by the agent as the values for the annotations *cpu-usage*, *memory-usage*, *net-usage*, and *disk-usage* of node n , respectively.

An AppGroup resource includes a *spec* property with three sub-properties: *runPeriod*, *deploymentSelector* and *rangeWidth*. The *runPeriod* property defines the interval between consecutive executions of the custom resource processing logic. The *deploymentSelector* property is used to identify the set of Deployments that make up a specific microservices-based application. These Deployments share the same *app-group* label but have distinct *app* labels corresponding to the microservices they are associated with. During each iteration, the agent retrieves the list of Deployments selected by the *deploymentSelector* property from the Kubernetes API server. For each Deployment d , the agent calculates the runtime usage of CPU, memory, network, and disk bandwidth resources, denoted as cpu_d , mem_d , net_d , and $disk_d$, respectively.

These values are derived from metrics gathered by the agent from the Prometheus metrics server. The Prometheus server collects these metrics from *CAdvisor*¹² agents, which

are deployed as DaemonSets on each cluster node to monitor resource usage metrics of the Pods running on that node. The metrics stored on the Prometheus server are time series collections of measurements, which the Prometheus server queries from the *CAdvisor* agents at a configurable interval of qt seconds.

Table 2 shows the *PromQL* queries executed by the agent on the Prometheus server to determine the runtime usage of each resource type for a Deployment d . Each value is calculated as the average resource usage across all Pods managed by Deployment d , specifically those running a container with the same name as Deployment d . Metrics are evaluated over a time duration equal to the *rangeWidth* property of the AppGroup custom resource. The Deployment d resource usage values are also the predicted ones for new Pods owned by that Deployment during scheduling.

The parameters cpu_d , mem_d , net_d , and $disk_d$ are then assigned by the agent as the values for the annotations *cpu-usage*, *memory-usage*, *net-usage*, and *disk-usage* of Deployment d , respectively.

Load-Aware Scheduler

The load-aware scheduler operates as a Deployment within the Kubernetes control plane and enhances the default Kubernetes scheduler by implementing a custom *LoadAware* scoring plugin written in Go. This plugin is built on the Kubernetes scheduler framework.

For each Pod scheduled, the plugin assigns a score to each candidate node in the cluster that has passed the filtering phase by executing a scoring function. The scores generated by the custom plugin are then combined with those from other scoring plugins within the default Kubernetes scheduler. During the scoring phase, the *Score()* function of the *LoadAware* scheduler plugin is invoked to assign a score to each cluster node n when scheduling a Pod p . Table 3 outlines the input parameters of the function.

The score of node n for the Pod p is determined by Equation (1):

$$score(p, n_i) = (1 - std(cpu_n, mem_n, net_n, disk_n)) \times MaxNodeScore \quad (1)$$

where:

$$MaxNodeScore = 100 \quad (2)$$

$$cpur_n = \frac{cpu_p + cpu_n}{totcpu_n} \quad (3)$$

$$memr_n = \frac{mem_p + mem_n}{totmem_n} \quad (4)$$

¹² <https://github.com/google/cadvisor>.

Table 1 Node metrics

Name	PromQL expression
cpu_n (mCore)	sum by (node_name) (rate(node_cpu_seconds_total{node_name=n,mode!=idle}[rangeWidth]))
mem_n (MB)	(avg_over_time(node_memory_MemTotal_bytes[rangeWidth])- avg_over_time(node_memory_MemFree_bytes[rangeWidth]))- avg_over_time(node_memory_Buffers_bytes[rangeWidth])- avg_over_time(node_memory_Cached_bytes[rangeWidth])) / (1024 * 1024)
net_n (MB/s)	sum by (node_name) (rate(node_network_transmit_bytes_total{node_name=n}[rangeWidth])+ rate(node_network_receive_bytes_total{node_name=n}[rangeWidth])) / (1024 * 1024)
$disk_n$ (MB/s)	sum by (node_name) (rate(node_disk_written_bytes_total{node_name=n}[rangeWidth])+ rate(node_disk_read_bytes_total{node_name=n}[rangeWidth])) / (1024 * 1024)

Table 2 Application metrics

Name	PromQL expression
cpu_d (mCore)	avg by(container) (rate(container_cpu_usage_seconds_total{container=d}[rangeWidth])) * 1000
mem_d (MB)	avg by(container) (avg_over_time(container_memory_working_set_bytes{container=d} [rangeWidth]) / (1024 * 1024))
net_d (MB/s)	avg by(container) (rate(container_network_transmit_bytes_total{container=d}[rangeWidth])+ rate(container_network_receive_bytes_total{container=d}[rangeWidth])) / (1024 * 1024)
$disk_d$ (MB/s)	avg by(container) (rate(container_fs_writes_bytes_total{container=d}[rangeWidth])+ rate(container_fs_reads_bytes_total{container=d}[rangeWidth])) / (1024 * 1024)

$$netr_n = \frac{net_p + net_n}{totnet_n} \quad (5)$$

$$diskr_n = \frac{disk_p + disk_n}{totdisk_n} \quad (6)$$

The resulting score is higher for nodes where the ratio between runtime usage and the total allocatable amount of various resource types is more balanced. The runtime usage of a resource type on a node is calculated as the sum of the predicted resource usage by Pod p and the total usage by all Pods running on that node. A lower standard deviation between the usage ratios of different resource types indicates more balanced resource utilization among the Pods on that node, leading to a higher node score.

The key rationale behind this scheduling strategy is to minimize shared resource interference among Pods, thereby reducing QoS violations related to application response times while simultaneously increasing cluster resource utilization by enabling more applications to be colocated on the same cluster. To achieve this, Pods that compete for the same resource type are distributed across different nodes whenever possible. Unlike the default Kubernetes scheduler plugins,

NodeResourcesFit and NodeResourcesBalancedAllocation, which assign node scores based on statically defined resource requirements set by application developers, the LoadAware scoring plugin utilizes real-time resource usage telemetry. This approach allows scheduling decisions to be dynamically adjusted based on the current resource usage patterns of Pods, which can vary over time.

Load-Aware Descheduler

The load-aware descheduler operates as a CronJob in the Kubernetes control plane and enhances the default Kubernetes descheduler by implementing a custom evictor plugin. This custom plugin is written in the Go language and is based on the Kubernetes descheduler framework.

The descheduler CronJob is configured to periodically run Jobs that execute the descheduling logic, comprising both the default evictor plugin and the proposed custom evictor plugin. The Job execution period is given by the *spec.runPeriod* property of the AppGroup custom resource associated with a microservices application. Within each iteration, the custom descheduler is configured to evict at maximum one Pod, while ensuring that at least one replica is maintained for each Deployment.

Table 3 Load-aware scheduler scoring function input parameters

Name	Description
cpu_p	The predicted runtime CPU usage of Pod p , as specified by the value of the <i>cpu-usage</i> annotation of the Deployment owning the Pod
mem_p	The predicted runtime memory usage of Pod p , as specified by the value of the <i>memory-usage</i> annotation of the Deployment owning the Pod
net_p	The predicted runtime network bandwidth usage of Pod p , as specified by the value of the <i>network-usage</i> annotation of the Deployment owning the Pod
$disk_p$	The predicted runtime disk bandwidth usage of Pod p , as specified by the value of the <i>disk-usage</i> annotation of the Deployment owning the Pod
cpu_n	The runtime total CPU usage on node n , as specified by the value of the <i>cpu-usage</i> annotation of the node
mem_n	The runtime total memory usage on node n , as specified by the value of the <i>memory-usage</i> annotation of the node
net_n	The runtime total network bandwidth usage on node n , as specified by the value of the <i>network-usage</i> annotation of the node
$disk_n$	The runtime total disk bandwidth usage on node n , as specified by the value of the <i>disk-usage</i> annotation of the node
$totcpu_n$	The total amount of allocatable CPUs on node n
$totmem_n$	The total amount of allocatable memory on node n
$totnet_n$	The total amount of allocatable network bandwidth on node n
$totdisk_n$	The total amount of allocatable disk bandwidth on node n

During each execution of the descheduler CronJob, the *Deschedule()* function of the custom eviction plugin is invoked. For each Pod p currently running in the node n_i the function assigns a score $dscore$ determined by Eq. (7):

$$dscore(p) = \max_{n_j \in cns} (score(p, n_j)) - score(p, n_i) \quad (7)$$

where $score(p, n_i)$ is the score assigned by the *Score()* function of the LoadAware scheduler plugin and *cns* is the set of cluster nodes. The Pod p selected for eviction is the one satisfying the conditions:

$$dscore(p) = \max_{p_j \in ps} (dscore(p, n_j)) \quad (8)$$

and

$$dscore(p) > 0 \quad (9)$$

where *ps* is the set of Pods running in the cluster.

This means that the Pod evicted is the one that would experience the greatest improvement in node score if it were to be rescheduled onto another node with a higher score than the one where it is currently executed. The configuration of the Default evictor plugin ensures that nodes where the Pod cannot be executed due to scheduling constraints are not considered as candidate nodes for migrating the Pod. This prevents the Pod from being unable to be rescheduled after eviction. Similar to the default Kubernetes descheduler, the proposed custom descheduler does not schedule replacements for evicted Pods; instead, it relies on the custom scheduler for that task.

The purpose of the proposed custom descheduler is to provide running Pods the opportunity to be rescheduled based on their runtime resource usage. This approach aims to optimize application placement at runtime. By evicting

currently running Pods and subsequently forcing them to be rescheduled, better placement decisions can be taken that help reduce the shared resource interference among Pods, enhancing the application response time.

Performance Evaluation

The proposed solution has been validated with microservices-based applications generated using the μ Bench benchmarking tool [13]. μ Bench enables the creation of service-mesh topologies comprising multiple microservices, each executing a designated function. One of the pre-built functions in μ Bench, known as the *Loader* function, simulates a generic workload that stresses node resources while

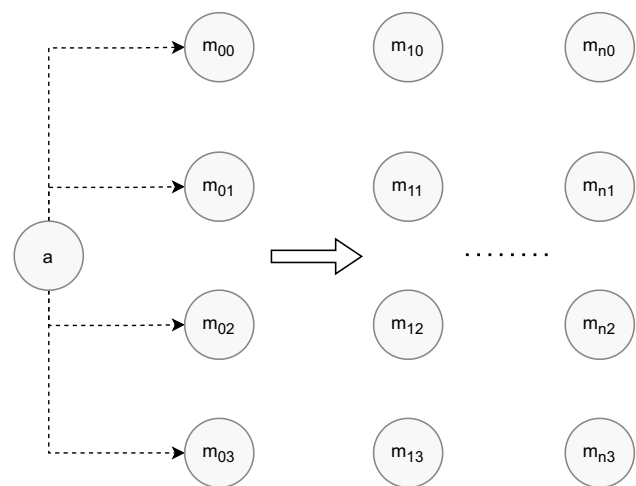
**Fig. 5** Generated microservices applications structure

Table 4 Loader functions configurations

Function	π decimals	Memory usage	Network bytes / req	Disk bytes / req
f_0	1000	64MB	1KB	1MB
f_1	100	512MB	1KB	1MB
f_2	100	64MB	40KB	1MB
f_3	100	64MB	1KB	10MB

processing HTTP requests. Upon invocation, the Loader function calculates N decimal places of π , with larger intervals leading to increased computational complexity and greater CPU load. Node memory and disk bandwidth stress can be customized by configuring the amount of memory usage and the number of disk read and write operations the function requires for each computation. Additionally, the function's network bandwidth usage can be configured by specifying the number of bytes returned in response to each request.

The microservices-based applications generated for evaluation are structured as illustrated in Fig. 5. Each application consists of a set of microservices distributed across N_l different layers, with each layer containing four microservices. A microservice $m_{i,j}$ belongs to layer i and runs a specific version of the Loader function f_j . Four versions of the Loader function are defined and listed in Table 4: f_0 , f_1 , f_2 , and f_3 . Each version is specifically configured to stress a different resource type, such as CPU, memory, network, or disk bandwidth. This approach models a realistic microservices-based application composed of a set of heterogeneous microservices, each competing for a specific type of resource. To reduce the impact of Pod rescheduling on the application availability each microservice runs two replicas.

The api gateway microservice a serves as the entry point for external user requests, which are then handled by backend microservices. These backend microservices interact with each other by exchanging HTTP requests. When the microservice a receives a request, it forwards the request to a microservice in the first layer. This process continues, with each layer's microservice forwarding the request to a microservice in the next layer until it reaches the last layer. Within each layer, the destination microservice is chosen randomly, with an equal distribution among the four available microservices.

The experimental testbed comprises a Rancher Kubernetes Engine 2 (RKE2)¹³ Kubernetes cluster, featuring one master node and six worker nodes. These nodes are deployed as virtual machines within a Proxmox¹⁴ environment, each

provisioned with 2 vCPUs, 8GB of RAM, a Gigabit virtual network adapter, and a virtual disk offering read and write bandwidth of approximately 500 MB/s.

Black box experiments are conducted to evaluate the end-to-end response time of the generated microservices applications when HTTP requests are sent to microservice a . A specified number of virtual users each send one request per second in parallel. Requests to the application are sent using the k6 load testing utility¹⁵ from a node within the same network as the cluster nodes, minimizing the impact of network latency on application response times.

Each experiment consists of five trials, during which the k6 tool sends requests to microservice a for 20 min. For each trial, statistics on the end-to-end application response time are collected and averaged with the results from the other trials in the same experiment. The experiments compare scenarios where our monitoring agent and the load-aware scheduler and descheduler components are deployed on the cluster against scenarios where only the default Kubernetes scheduler is used. The Cluster and AppGroup custom resources used to configure the monitoring agent and the load-aware descheduler are shown in Listing 2. The Prometheus server collects telemetry data from the node exporters and CAdvisor agents every 10 s.

Two scenarios are considered. In the first scenario, the response time of a single application is analyzed with respect to the number of layers N_l that constitute the application. This set of experiments aims to demonstrate the effectiveness of our approach in mitigating the impact of application complexity and the competition among microservices for shared resources on the application's response time. In the second scenario, the response time of multiple identical single-layer applications is analyzed based on the number of applications deployed on the same cluster. This set of experiments aims to demonstrate the effectiveness of our approach in enhancing resource utilization by enabling the colocation of multiple applications on the same cluster, while maintaining response times close to those observed in single-application deployments.

Figure 6 shows the results from the first scenario experiments, where each graph corresponds to a different number of application layers N_l being analyzed. These graphs provide a visual representation of the 95th percentile of the application response time, plotted against the number of virtual users concurrently sending requests to the application.

The proposed approach consistently outperforms the default Kubernetes scheduler across all tested scenarios, showing average response time improvements of 13%, 30%, 42% and 53% for varying numbers of application layers. For a low number of virtual users, the proposed approach

¹³ <https://docs.rke2.io>.

¹⁴ <https://www.proxmox.com>.

¹⁵ <https://k6.io>.

yields performance similar to that of the default scheduler. This similarity is primarily due to the limited interference among Pods that are colocated on the same nodes by the default scheduler, resulting in minimal contention for shared resources.

However, as the number of virtual users increases, the advantages of the proposed approach become more pronounced. The response times under the default Kubernetes scheduler begin to increase more rapidly, leading to significant delays. In contrast, the proposed approach demonstrates a more gradual increase in response times, maintaining better performance. This disparity in response times becomes particularly evident at higher virtual user counts, where the proposed approach exhibits increasingly superior performance.

Additionally, the gap between the response times of the proposed approach and the default scheduler grows wider with a higher number of application layers. This indicates that the proposed approach not only improves response times under heavier loads but also scales more effectively with application complexity, providing substantial benefits for multi-layered applications.

Figure 7 shows the results from the second scenario experiments, where each graph corresponds to a different number of applications N_a , each with a single layer, colocated on the same cluster. These graphs provide a visual representation of the average 95th percentile of the application's response times, plotted against the number of virtual users concurrently sending requests to the application. The response times are compared with those obtained with our proposed approach for a single-deployment application. In this case as well, the proposed approach consistently outperforms the default Kubernetes scheduler across all tested scenarios. With the default scheduler, the percentage differences in response times compared to those of a single-deployment application are 35%, 107%, 166%, and 233%, depending on the number of application layers. In contrast, our approach reduces these differences to 20%, 43%, 72%, and 106%, respectively. Similar observations can be made for this second scenario: as the application load increases with more users and the number of application layers grows, the performance of our approach improves significantly compared to the default Kubernetes scheduler.

Related Work

In the literature, several studies have proposed extending the Kubernetes platform to overcome the limitations of its static orchestration strategy, particularly when applied to microservices-based applications that share the same node cluster [14–16].

An approach for scheduling workloads in a Kubernetes cluster, particularly when they are unevenly distributed or subject to fluctuations in resource utilization, is presented in [17]. This approach proposes a hybrid shared-state scheduling framework, where most tasks are delegated to distributed scheduling agents. It also incorporates a scheduling correction function to manage unscheduled and low-priority tasks. Scheduling decisions are informed by the cluster's overall state, which is periodically synchronized and updated by a master-state agent. While real-time resource usage metrics are considered, similar to our approach, to reduce shared resource interference between services, the evaluation is limited to CPU and memory resources, excluding network and disk bandwidth.

In [18], a multiobjective approach is proposed to enhance the response time of applications running on Kubernetes clusters by minimizing both inter-server network traffic and CPU throttling on overloaded servers. This approach employs a Simulated Annealing combinatorial heuristic algorithm to solve the multiobjective optimization problem. Also in this work, the evaluation of resource usage is limited to specific resources, namely CPU and network bandwidth, while memory and disk bandwidth are not taken into account.

NetMARKS [19] is a Kubernetes scheduler extender that uses dynamic network metrics collected with Istio Service Mesh to ensure efficient placement of Service Function Chains, based on the historical traffic amount exchanged between services. However, the proposed approach does not account for real-time cluster network conditions during scheduling decisions, nor does it reschedule services if the initial placement becomes suboptimal over time. In contrast, our approach considers both application-level and infrastructure-level network bandwidth usage to periodically adjust and optimize service placement.

In [20], Nautilus is introduced as a run-time system that incorporates a contention-aware resource manager and a load-aware microservice scheduler. The resource manager, deployed on each node, leverages reinforcement learning to optimize resource allocation for microservices, addressing complex contention patterns. Simultaneously, the microservice scheduler continuously monitors the system's quality of service (QoS) and dynamically migrates microservices from overutilized nodes to underutilized ones during runtime. Similar to our work, both application-level and infrastructure-level resource usage metrics are considered to determine workload placement decisions and to migrate them in case of performance degradation.

Zeus [21] is a highly scalable cluster scheduling system built using Kubernetes extension mechanisms. It enables the safe colocation of best-effort jobs alongside latency-sensitive services. Zeus schedules best-effort jobs based on actual server utilization and can dynamically allocate

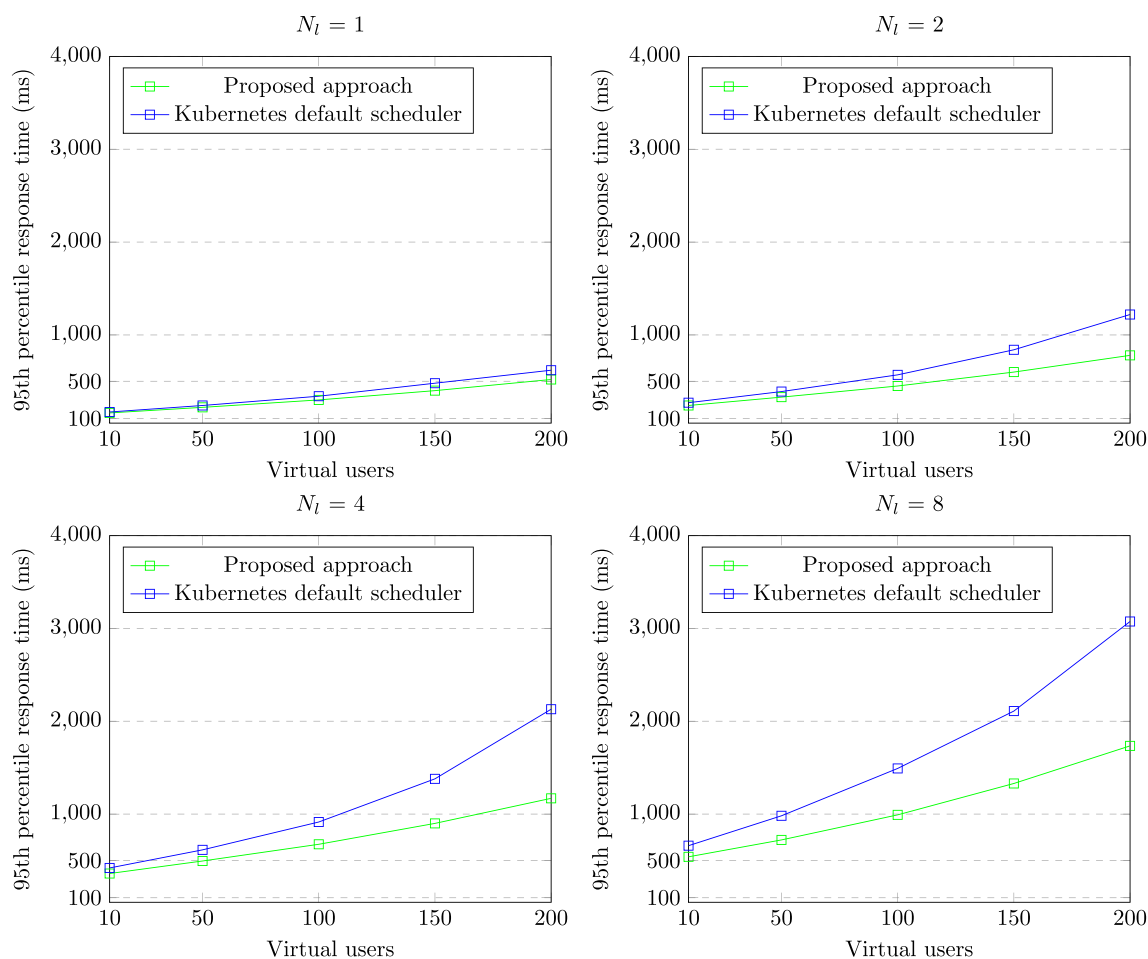


Fig. 6 First scenario experiments results

resources between these two workload classes. Additionally, it enhances container isolation by coordinating both software and hardware isolation features. Consequently, Zeus significantly improves the resource utilization of Kubernetes clusters. However, while dynamic workload resource allocation is performed in the proposed approach, no rescheduling actions are taken to mitigate shared resource interference between workloads and to enhance their performance.

In [22], DRS is proposed—a deep reinforcement learning-enhanced Kubernetes scheduler aimed at addressing issues of resource fragmentation and low utilization caused by the default Kubernetes scheduler’s inefficiencies. The scheduling problem is framed as a Markov decision process, with specifically designed state, action, and reward structures to improve resource usage and reduce load imbalance. A DRS monitor is introduced to track parameters related to resource utilization and provide a comprehensive view of global resource availability. Ultimately, DRS is designed to autonomously refine its scheduling policy through interaction with the Kubernetes cluster, eliminating the need for expert

knowledge about workload and cluster conditions. Also in this work no rescheduling actions are taken to mitigate shared resource interference among workloads.

Boreas [23] is a Kubernetes scheduler designed to handle simultaneous bursts of deployment requests. It determines the optimal placement of service containers while considering deployment constraints and runtime containers resource requirements through the use of a configuration optimizer. However, although the proposed scheduler evaluates the dynamic resource requirements of workloads, it does not account for real-time resource availability on infrastructure nodes.

In [24] Diktyo is presented, a network-aware scheduling framework for the Kubernetes platform, inspired by its scheduling plugins architecture. Diktyo proposes additional scheduling plugins aiming to determine low-latency deployment schemes for applications scheduled on Kubernetes clusters. The framework minimizes the application’s end-to-end latency by selecting nodes with low network costs for dependent microservices and with sufficient bandwidth capacity based on previous microservice allocations.

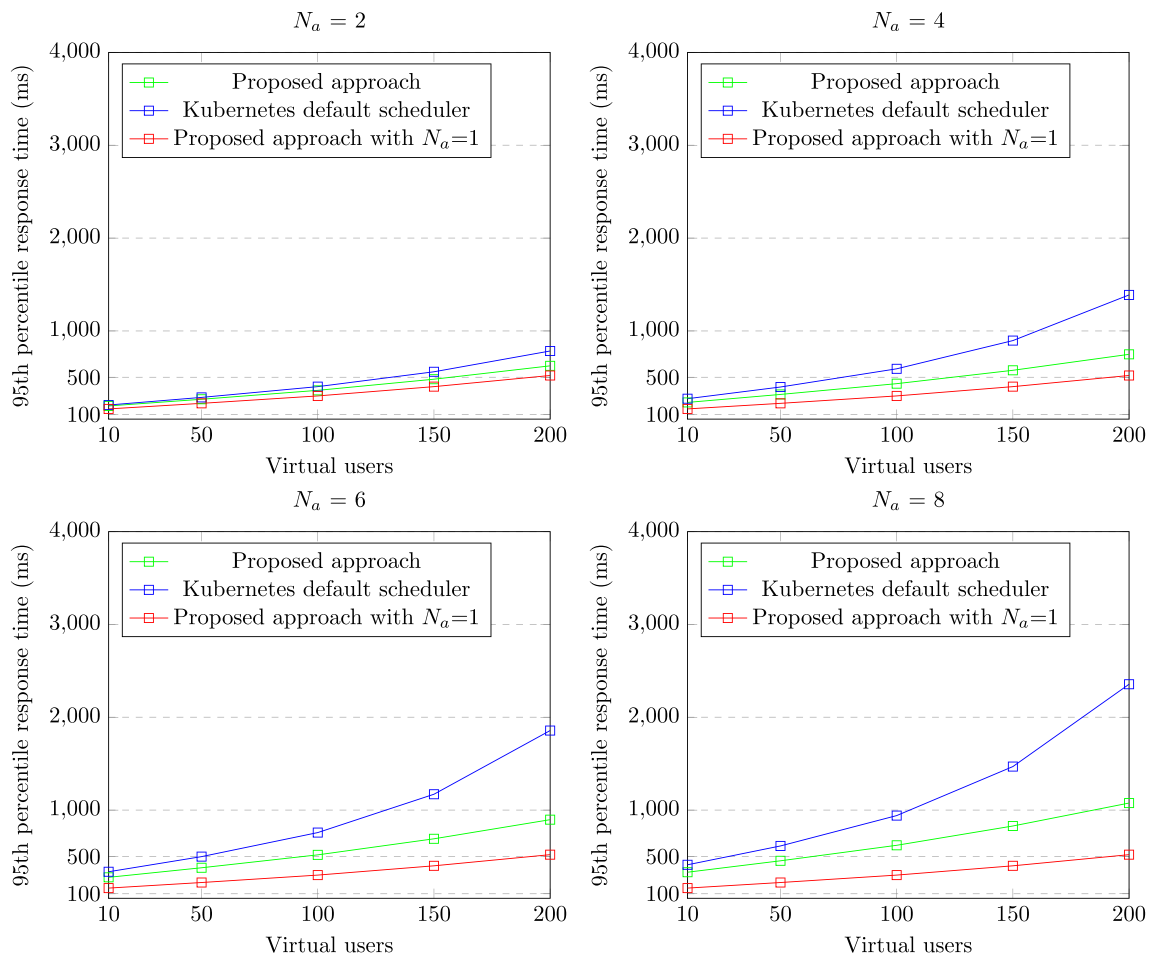


Fig. 7 Second scenario experiments results

However in this work, there is no service rescheduling if network conditions change over time.

In [25], a dynamic resource management and provisioning scheme for Kubernetes infrastructure is introduced. This approach enables the real-time adjustment of resource allocation for Pods, effectively mitigating the limitations associated with traditional resource restriction challenges. While in-place adjustments to resource allocation for Pods are performed, no rescheduling actions are undertaken to enhance workload performance.

Finally, in our previous works [26] and [27], we proposed a network-aware Kubernetes scheduler designed to reduce network distance between microservices with high communication needs, thereby improving application response time. The load-aware scheduler plugin introduced in this work complements the network-aware scheduler, and both can be integrated and used together within the same scheduling framework.

Conclusions and Future Work

In this work, we propose extending the Kubernetes platform with a real-time, load-aware orchestration strategy. Our approach aims to minimize shared resource interference among distributed, microservices-based applications running on the same cluster, thereby reducing QoS violations related to response times. At the same time, it enhances cluster resource utilization by enabling the colocation of multiple applications while maintaining acceptable application performance. The primary objective is to address the limitations of Kubernetes' static scheduling and descheduling policies, which depend on pre-determined knowledge of each microservice's resource requirements for optimal container placement and rescheduling decisions. Given the dynamic nature of distributed microservices, we propose augmenting the Kubernetes scheduler and descheduler with custom plugins that leverage real-time telemetry data on infrastructure and application resource usage. This strategy reduces reliance on static resource profiling while ensuring

that applications consistently meet their performance expectations. Results demonstrate that our approach outperforms the default Kubernetes scheduler and descheduler implementations in two key areas: reducing response times for individual applications and increasing the number of applications that can be colocated on the same cluster while still maintaining acceptable response times.

As future work, we plan to enhance the proposed scheduling and descheduling strategies by using AI and time series analysis techniques. This will enable the design of proactive algorithms that consider long-term telemetry data to improve predictions of both application and infrastructure runtime states, thereby allowing for more optimal scheduling and descheduling decisions.

Funding Open access funding provided by Università degli Studi di Catania within the CRUI-CARE Agreement. This work was partially funded by the European Union under the Italian National Recovery and Resilience Plan (NRRP) of NextGenerationEU, Mission 4 Component C2 Investment 1.1 - Call for tender No. 1409 of 14/09/2022 of Italian Ministry of University and Research - Project "Cloud Continuum aimed at On-Demand Services in Smart Sustainable Environments" - CUP E53D23016420001.

Data availability The authors declare that the data supporting the findings of this study are available within the paper. Should any raw data files be needed in another format they are available from the corresponding author upon reasonable request.

Declarations

Conflict of interest The authors declare that they have no Conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Salaht FA, Desprez F, Lebre A. An overview of service placement problem in fog and edge computing. *ACM Comput Surv.* 2020; 53(3) <https://doi.org/10.1145/3391196>.
2. Oleghe O. Container placement and migration in edge computing: Concept and scheduling models. *IEEE Access.* 2021;9:68028–43. <https://doi.org/10.1109/ACCESS.2021.3077550>.
3. Luo Q, Hu S, Li C, Li G, Shi W. Resource scheduling in edge computing: a survey. *CoRR abs/2108.08059*. 2021 [arXiv: 2108.08059](https://arxiv.org/abs/2108.08059).
4. Varghese B, Lara E, Ding A, Hong C, Bonomi F, Dustdar S, Harvey P, Hewkin P, Shi W, Thiele M, Willis P. Revisiting the arguments for edge computing research. *IEEE Internet Comput.* 2021;25(05):36–42. <https://doi.org/10.1109/MIC.2021.3093924>.
5. Kong X, Wu Y, Wang H, Xia F. Edge computing for internet of everything: a survey. *IEEE Internet Things J.* 2022;9(23):23472–85. <https://doi.org/10.1109/JIOT.2022.3200431>.
6. Khan WZ, Ahmed E, Hakak S, Yaqoob I, Ahmed A. Edge computing: a survey. *Futur Gener Comput Syst.* 2019;97:219–35. <https://doi.org/10.1016/j.future.2019.02.050>.
7. Kayal P. Kubernetes in fog computing: Feasibility demonstration, limitations and improvement scope : Invited paper. In: 2020 IEEE 6th World Forum on Internet of Things (WF-IoT), 2020; 1–6. <https://doi.org/10.1109/WF-IoT48130.2020.9221340>.
8. Manaouil K, Lebre A. Kubernetes and the Edge? Research Report RR-9370, Inria Rennes - Bretagne Atlantique (October 2020). <https://hal.inria.fr/hal-02972686>.
9. Goudarzi M, Palaniswami M, Buyya R. Scheduling iot applications in edge and fog computing environments: a taxonomy and future directions. *ACM Comput. Surv.* 2022;55(7) <https://doi.org/10.1145/3544836>.
10. Burns B, Grant B, Oppenheimer D, Brewer E, Wilkes J. Borg, omega, and kubernetes: lessons learned from three container-management systems over a decade. *Queue.* 2016;14(1):70–93. <https://doi.org/10.1145/2898442.2898444>.
11. Marchese A, Tomarchio O. Load-Aware Container Orchestration on Kubernetes Clusters. In: Proceedings of the 14th International conference on cloud computing and services science (CLOSER 2024), pp. 92–102. SciTePress, Angers, France (2024). <https://doi.org/10.5220/0012738800003711>.
12. Gannon D, Barga R, Sundaresan N. Cloud-native applications. *IEEE Cloud Comput.* 2017;4:16–21. <https://doi.org/10.1109/MCC.2017.4250939>.
13. Detti A, Funari L, Petrucci L. μ bench: an open-source factory of benchmark microservice applications. *IEEE Trans Parallel Distrib Syst.* 2023;34(3):968–80. <https://doi.org/10.1109/TPDS.2023.3236447>.
14. Carrión C. Kubernetes scheduling: taxonomy, ongoing issues and challenges. *ACM Comput Surv.* 2022;55(7) <https://doi.org/10.1145/3539606>.
15. Rejiba Z, Chamanara J. Custom scheduling in kubernetes: a survey on common problems and solution approaches. *ACM Comput Surv.* 2022;55(7) <https://doi.org/10.1145/3544788>.
16. Senjab K, Abbas S, Ahmed N, Khan AR. A survey of kubernetes scheduling algorithms. *J Cloud Comput.* 2023;12(1):87. <https://doi.org/10.1186/s13677-023-00471-1>.
17. Ungureanu O-M, Vlădeanu C, Kooij R. Kubernetes cluster optimization using hybrid shared-state scheduling framework. In: Proceedings of the 3rd International Conference on Future Networks and Distributed Systems. ICFNDS '19. Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3341325.3341992>.
18. Bracke V, Santos J, Wauters T, De Turck F, Volckaert B. A multi-objective metaheuristic-based container consolidation model for cloud application performance improvement. *J Netw Syst Manage.* 2024;32(3):61. <https://doi.org/10.1007/s10922-024-09835-7>.
19. Wojciechowski L, Opasiak K, Latusek J, Wereski M, Morales V, Kim T, Hong M. NetMARKS: Network Metrics-AwaRe Kubernetes Scheduler Powered by Service Mesh. In: IEEE INFOCOM 2021 - IEEE Conference on Computer Communications, 2021; 1–9. <https://doi.org/10.1109/INFOCOM42981.2021.9488670>.
20. Fu K, Zhang W, Chen Q, Zeng D, Peng X, Zheng W, Guo M. Qos-aware and resource efficient microservice deployment in cloud-edge continuum. In: IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2021; 932–941. <https://doi.org/10.1109/IPDPS49936.2021.00102>.

21. Zhang X, Li L, Wang Y, Chen E, Shou L. Zeus: improving resource efficiency via workload colocation for massive Kubernetes clusters. *IEEE Access*. 2021;9:105192–204. <https://doi.org/10.1109/ACCESS.2021.3100082>.
22. Jian Z, Xie X, Fang Y, Jiang Y, Lu Y, Dash A, Li T, Wang G. DRS: a deep reinforcement learning enhanced Kubernetes scheduler for microservice-based system. *Softw Pract Exp*. 2023;54(10):2102–26. <https://doi.org/10.1002/spe.3284>.
23. Lebesbye T, Mauro J, Turin G, Yu IC. Boreas - a service scheduler for optimal Kubernetes deployment. In: Hacid H, Kao O, Mecella M, Moha N, Paik H-Y, editors. *Service-oriented computing*. Cham: Springer; 2021. p. 221–37.
24. Santos J, Wang C, Wauters T, De Turck F. Diktyo: network-aware scheduling in container-based clouds. *IEEE Trans Netw Serv Manage*. 2023;20(4):4461–77. <https://doi.org/10.1109/TNSM.2023.3271415>.
25. Kim J, No J, Park S-s. Effective resource provisioning scheme for kubernetes infrastructure. In: Nagar AK, Jat DS, Mishra D, Joshi A (eds.) *Intelligent Sustainable Systems*, pp. 75–85. Springer, Singapore (2024).
26. Marchese A, Tomarchio O. Network-Aware Container Placement in Cloud-Edge Kubernetes Clusters. In: *2022 22nd IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, Taormina, Italy, 2022;859–865. <https://doi.org/10.1109/CCGrid54584.2022.00102>.
27. Marchese A, Tomarchio O. Extending the Kubernetes Platform with Network-Aware Scheduling Capabilities. In: *Service-Oriented Computing: 20th International Conference, ICSOC 2022, Seville, Spain, November 29 - December 2, 2022, Proceedings*, pp. 465–480. Springer, Seville, Spain (2022). https://doi.org/10.1007/978-3-031-20984-0_33.

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.